noahabe rename: added chapter titles to folder names 93005ad · 2 years ago

> Using the university schema, write an SQL query to find section(s) with maximum enrollment. The result columns should appear in the order "courseid, secid, year, semester, num". (It may be convenient to use the *with* construct.)

```
WITH section_student_frequency(courseid, secid, year, semester, num) AS
    SELECT course_id, sec_id, year, semester, COUNT(DISTINCT ID)
    FROM takes
    GROUP BY course_id, sec_id, year, semester
)
SELECT *
FROM section_student_frequency
WHERE num = (SELECT MAX(num) FROM section_student_frequency);
```

noahabe rename: added chapter titles to folder names 93005ad · 2 years ago

36 lines (29 loc) · 1.45 KB

> Consider the insurance database of Figure 3.17, where the primary keys are underlined. Construct the following SQL queries for this relational database.
> a. Find the total number of people who owned cars that were involved in accidents in 2017.
> b. Delete all year-2010 cars belonging to the person whose ID is '12345'.

---

a. Find the total number of people who owned cars that were involved in accidents in 2017.

Note: This is not the same as the total number of accidents in 2017. We must count people with several accidents only once. Furthermore, note that the question asks for owners, and it might be that the owner of the car was not the driver actually involved in the accident.

```
SELECT COUNT(DISTINCT person.driver_id)
FROM accident, participated, person, owns
WHERE accident.report_number = participated.report_number
      AND owns.driver_id = person.driver_id
      AND owns.license_plate = participated.license_plate
      AND year = 2017
```

b. Delete all year-2010 cars belonging to the person whose ID is '12345'.

```
DELETE FROM car
WHERE year = 2010 AND license_plate IN
    (SELECT license_plate FROM owns WHERE driver_id = '12345')
```

Note: The *owns*, *accident* and *participated* records associated with the deleted cars still exist. (If you want the corresponding rows in *owns* and *participated* to also be deleted you can use **ON CASCADE DELETE** on the schema of the tables).

noahabe rename: added chapter titles to folder names 93005ad · 2 years ago

11 lines (9 loc) · 410 Bytes

Preview Code Blame Raw

> Using the university schema, write an SQL query to find the number of students in each section. The result columns should appear in the order "course_id, sec_id, year,semester,num". You do not need to output sections with 0 students.

```sql
SELECT course_id, sec_id, year, semester, COUNT(DISTINCT ID) AS num
FROM takes
GROUP BY course_id, sec_id, year, semester;
```

noahabe rename: added chapter titles to folder names 93005ad · 2 years ago

17 lines (15 loc) · 548 Bytes

Preview Code Blame Raw

> Using the university schema, write an SQL query to find the ID and title of each course in Comp. Sci. that has had at least one section with afternoon hours (i.e., ends at or after 12:00). (You should eliminate duplicates if any.)

```
SELECT course_id,title
FROM course AS c
WHERE dept_name = 'Comp. Sci.' AND
    EXISTS (
        SELECT *
        FROM section
        WHERE section.course_id = c.course_id AND
        time_slot_id IN (SELECT time_slot_id FROM time_slot WHERE end_h
    )
```

noahabe rename: added chapter titles to folder names 93005ad · 2 years ago

29 lines (25 loc) · 1005 Bytes

Preview Code Blame Raw

> Rewrite the preceding query, but also ensure that you include only instructors who have given at least one other non-null grade in some course.

So the question paraphrased is as follows: write an SQL query to find the ID and name of each instructor who has never given an A grade in any course she or he has taught and who has given at least one other non-null grade in some course.

```
SELECT id, name
FROM instructor AS i
WHERE 'A' NOT IN (
    SELECT takes.grade
    FROM takes INNER JOIN teaches
        ON (takes.course_id,takes.sec_id,takes.semester,takes.year) =
           (teaches.course_id,teaches.sec_id,teaches.semester,teaches.y
    WHERE teaches.id = i.id
)
AND
(
    SELECT COUNT(*)
    FROM takes INNER JOIN teaches
        ON (takes.course_id,takes.sec_id,takes.semester,takes.year) =
           (teaches.course_id,teaches.sec_id,teaches.semester,teaches.y
    WHERE teaches.id = i.id AND takes.grade IS NOT NULL
) >= 1
```

noahabe rename: added chapter titles to folder names 93005ad · 2 years ago

135 lines (114 loc) · 4.2 KB

> Write the following queries in SQL, using the university schema. (We suggest you actually run these queries on a database, using the sample data that we provide on the website of the book, db-book.com. Instructions for setting up a database, and loading sample data, are provided on the above website.)
>
> a. Find the titles of courses in the Comp. Sci. department that have 3 credits.
> b. Find the IDs of all students who were taught by an instructor named Einstein; make sure there are no duplicates in the result.
> c. Find the highest salary of any instructor.
> d. Find all instructors earning the highest salary (there may be more than one with the same salary).
> e. Find the enrollment of each section that was offered in Fall 2017.
> f. Find the maxium enrollment, across all sections, in Fall 2017.
> g. Find the sections that had the maximum enrollment in Fall 2017.

a. Find the titles of courses in the Comp. Sci. department that have 3 credits.

```
SELECT title
FROM course
WHERE dept_name = 'Comp. Sci.' AND credits = 3;
```

b. Find the IDs of all students who were taught by an instructor named Einstein; make sure there are no duplicates in the result. This query can be answered in several different ways. One way is as follows.

```
SELECT DISTINCT takes.ID
FROM takes, instructor, teaches
WHERE takes.course_id = teaches.course_id AND
    takes.sec_id = teaches.sec_id AND
    takes.semester = teaches.semester AND
    takes.year = teaches.year AND
    teaches.id = instructor.id AND
    instructor.name = 'Einstein'
```

Another method by using subqueries

```
SELECT DISTINCT id
FROM takes
WHERE (course_id, sec_id, semester, year) IN
(
    SELECT course_id, sec_id, semester, year
    FROM teaches INNER JOIN instructor
        ON teaches.id = instructor.id
    WHERE instructor.name = 'Einstein'
);
```

c. Find the highest salary of any instructor.

```
SELECT MAX(salary)
FROM instructor
```

d. Find all instructors earning the highest salary (there may be more than one with the same salary).

```
SELECT id, name
FROM instructor
WHERE salary = (SELECT MAX(salary) FROM instructor)
```

e. Find the enrollment of each section that was offered in Fall 2017.

```
SELECT course_id, sec_id, (
    SELECT COUNT(id)
    FROM takes
    WHERE takes.year = section.year
        AND takes.semester = section.semester
        AND takes.course_id = section.course_id
        AND takes.sec_id = section.sec_id
) AS enrollment
FROM section
WHERE semester = 'Fall' AND year = 2017
```

Note that if the result of the subquery is empty, the aggregate function **count** returns a value of 0. One way of writing the query might appear to be:

```
SELECT course_id, sec_id, COUNT(id)
FROM takes
WHERE semester = 'Fall' AND year = 2017
GROUP BY course_id, sec_id
```

But note that if a section does not have any students taking it, it would not appear in the result. One way of ensuring such a section appears with a count of 0 is to use the **outer join** operation, covered in Chapter 4.

f. Find the maxium enrollment, across all sections, in Fall 2017. One way of writing this query is as follows:

```
WITH enrollment_in_fall_2017(course_id, sec_id, enrollment) AS (
    SELECT course_id, sec_id, COUNT(id)
    FROM takes
    WHERE semester = 'Fall' AND year = 2017
    GROUP BY course_id, sec_id
)
SELECT CASE
        WHEN MAX(enrollment) IS NOT NULL THEN MAX(enrollment)
        ELSE 0
       END
FROM enrollment_in_fall_2017;
```

g. Find the sections that had the maximum enrollment in Fall 2017.

```
WITH enrollment_in_fall_2017(course_id, sec_id, enrollment) AS (
    SELECT course_id, sec_id, COUNT(id)
    FROM takes
    WHERE semester = 'Fall' AND year = 2017
    GROUP BY course_id, sec_id
)
SELECT course_id, sec_id
FROM enrollment_in_fall_2017
WHERE enrollment = (SELECT MAX(enrollment) FROM enrollment_in_fall_2017
```

It is also possible to write the query without the **with** clause, but the subquery to find enrollment would get repeated twice in the query. While not incorrect to add **distinct** in the **count**, it is not necessary in light of the primary key constraint on *takes*.

noahabe rename: added chapter titles to folder names 93005ad · 2 years ago

118 lines (102 loc) · 4.41 KB

> Suppose you are given a relation *grade_points(grade, points)* that provides a conversion from letter grades in the *takes* relation to numeric scores; for example, an "A" grade could be specified to correspond to 4 points, an "A-" to 3.7 points, a "B+" to 3.3 points, a "B" to 3 points and so on. The grade points earned by a student for a course offering (section) is defined as the number of credits for the course multiplied by the numeric points for the grade that the student received.
>
> Given the preceding relation and our university schema, write each of the following queries in SQL. You may assume for simplicity that no *takes* tuple has the *null* value for *grade*.
> a. Find the total grade points earned by the student with ID '12345', across all courses taken by the student.
> b. Find the grade point average (GPA) for the above student, that is, the total grade points divided by the total credits for the associated courses.
> c. Find the ID and the grade-point average of each student.
> d. Now reconsider your answers to the earlier parts of this exercise under the assumption that some grades might be null. Explain whether your solutions still work and, if not, provide versions that handle nulls properly.

---

a. Find the total grade points earned by the student with ID '12345', across all courses taken by the student.

```
SELECT SUM(c.credits * g.points)
FROM takes AS t
    INNER JOIN course AS c ON t.course_id = c.course_id
    INNER JOIN grade_points AS g ON t.grade = g.grade
WHERE t.ID = '12345'
```

In the above query, a student who has not taken any course would not have any tuples, whereas we would expect to get 0 as the answer. One way of fixing this problem is to use the **outer join** operation, which we study later in Chapter 4. Another way to ensure that we get 0 as the answer is via the following query:

```
(
SELECT SUM(c.credits * g.points)
FROM takes AS t
    INNER JOIN course AS c ON t.course_id = c.course_id
    INNER JOIN grade_points AS g ON t.grade = g.grade
WHERE t.ID = '12345'
)
UNION
(
SELECT 0
FROM student
WHERE ID = '12345' AND NOT EXISTS (SELECT * FROM takes WHERE ID = '1234
)
```

b. Find the grade point average (GPA) for the above student, that is, the total grade-points divided by the total credits for the associated courses.

```
SELECT SUM(c.credits * g.points) / SUM(credits) AS GPA
FROM takes AS t
    INNER JOIN course AS c ON t.course_id = c.course_id
    INNER JOIN grade_points AS g ON t.grade = g.grade
WHERE t.ID = '12345'
```

As before, a student who has not taken any course would not appearn in the above result; we can ensure that such a stuent appears in the result by using the modified query from the previous part of this question. However, an additional issue in this case is that the sum of credits would also be 0, resulting in a divide-by-zero condition. In fact, the only meaningful way of defining the GPA in this case is to define it as *null*. We can ensure that such a student appears in the result with a null *GPA* by adding the following **union** clause to the above query.

```
UNION
(
    SELECT null AS GPA
    FROM student
    WHERE ID = '12345' AND
        NOT EXISTS (SELECT * FROM takes WHERE ID='12345')
)
```

c. Find the ID and the grade-point average of each student.

```
SELECT t.ID, SUM(c.credits * g.points) / SUM(credits) AS GPA
FROM takes AS t
    INNER JOIN course AS c ON t.course_id = c.course_id
    INNER JOIN grade_points AS g ON t.grade = g.grade
GROUP BY t.ID
```

Again, to handle students who have not taken any course, we would have to add the following **union** clause:

```
UNION
(
    SELECT ID, null AS GPA
    FROM student
    WHERE NOT EXISTS (SELECT * FROM takes WHERE takes.ID = student.ID)
)
```

d. Now reconsider your answers to the earlier parts of this exercise under the assumption that some grades might be null. Explain whether your solutions still work and, if not, provide versions that handle nulls properly.

The queries listed above all include a test of equality on *grade* between *grade_points* and *takes*. Thus, for any *takes* tuple with a *null* grade, that student's course would be eliminated from the rest of the computation of the result. As a result, the credits of such courses would be eliminated also, and thus the queries would return the correct answer even if some grades are null.

noahabe rename: added chapter titles to folder names 93005ad · 2 years ago

32 lines (25 loc) · 1.04 KB

> Write the following inserts, deletes, or updates in SQL, using the university schema.
> a. Increase the salary of each instructor in the Comp. Sci. department by 10%.
> b. Delete all courses that have never been offered (i.e., do not occur in the *section* relation).
> c. Insert every student whose *tot_cred* attribute is greater than 100 as an instructor in the same department, with a salary of $10,000.

---

a. Increase the salary of each instructor in the Comp. Sci. department by 10%.

```
UPDATE instructor
SET salary = salary * 1.10
WHERE dept_name = 'Comp. Sci.'
```

b. Delete all courses that have never been offered (i.e., do not occur in the *section* relation).

```
DELETE FROM course
WHERE course_id NOT IN (SELECT course_id FROM section)
```

c. Insert every student whose *tot_cred* attribute is greater than 100 as an instructor in the same department, with a salary of $10,000.

```
INSERT INTO instructor
SELECT ID, name, dept_name, 10000
FROM student
WHERE tot_cred > 100
```

noahabe rename: added chapter titles to folder names 93005ad · 2 years ago

43 lines (37 loc) · 1.16 KB

Preview Code Blame Raw

> Suppose that we have a relation *marks(ID,score)* and we wish to assign grades to students based on the score as follows: grade *F* if *score* < 40, grade *C* if 40 ≤ *score* < 60, grade *B* if 60 ≤ *score* < 80, and grade *A* if 80 ≤ *score*. Write SQL queries to do the following:
>
> a. Display the grade for each student, based on the *marks* relation.
> b. Find the number of students with each grade.

---

a. Display the grade for each student, based on the *marks* relation.

```
SELECT ID,
    CASE
        WHEN score < 40 THEN 'F'
        WHEN score < 60 THEN 'C'
        WHEN score < 80 THEN 'B'
        ELSE 'A'
    END
FROM marks
```

b. Find the number of students with each grade.

```
WITH grades(ID,grade) AS (
    SELECT ID,
        CASE
            WHEN score < 40 THEN 'F'
            WHEN score < 60 THEN 'C'
            WHEN score < 80 THEN 'B'
            ELSE 'A'
        END
    FROM marks
)
SELECT grade, COUNT(ID)
FROM grades
GROUP BY grade
```

As an alternative, the **WITH** clause can be removed, and instead the definition of *grades* can be made a subquery of the main query.

noahabe rename: added chapter titles to folder names 93005ad · 2 years ago

12 lines (10 loc) · 404 Bytes

Preview Code Blame Raw

> The SQL **LIKE** operator is case sensitive (in most systems), but the **lower()** function on strings can be used to perform case-insensitive matching. To show how, write a query that finds departments whose names contain the string "sci" as a substring, regardless of the case.

```
SELECT dept_name
FROM department
WHERE LOWER(dept_name) LIKE '%sci%'
```

noahabe rename: added chapter titles to folder names 93005ad · 2 years ago

> Consider the SQL query

```
SELECT p.a1
FROM p, r1, r2
WHERE p.a1 = r1.a1 OR p.a1 = r2.a1
```

> Under what conditions does the preceding query select values of *p.a1* that are either in *r1* or in *r2* ? Examine carefully the cases where either *r1* or *r2* may be empty.

The query selects those values of *p.a1* that are equal to some value of *r1.a1* or *r2.a1* if and only if both *r1* and *r2* are non-empty. If one or both of *r1* and *r2* are empty, the Cartesian product of *p*, *r1* and *r2* is empty, hence the result of the query is empty. If *p* itself is empty, the result is empty.

noahabe rename: added chapter titles to folder names 93005ad · 2 years ago

48 lines (39 loc) · 1.49 KB

Preview Code Blame Raw

> Consider the bank database of Figure 3.18, where the primary keys are underlined. Construct the following SQL queries for this relational database.
> a. Find the ID of each customer of the bank who has an account but not a loan.
> b. Find the ID of each customer who lives on the same street and in the same city as customer '12345'.
> c. Find the name of each branch that has at least one customer who has an account in the bank and who lives in "Harrison".

---

a. Find the ID of each customer of the bank who has an account but not a loan.

```
(SELECT ID FROM depositor)
EXCEPT
(SELECT ID FROM borrower)
```

b. Find the ID of each customer who lives on the same street and in the same city as customer '12345'

```
SELECT F.ID
FROM customer AS F, customer AS S
WHERE F.customer_street = S.customer_street
    AND F.customer_city = S.customer_city
    AND S.customer_id = '12345';
```

Another method (using **scalar subqueries**)

```
SELECT ID
FROM customer
WHERE customer_street = (SELECT customer_street FROM customer WHERE ID
    customer_city = (SELECT customer_city FROM customer WHERE ID = '123
```

c. Find the name of each branch that has at least one customer who has an account in the bank and who lives in "Harrison".

```
SELECT DISTINCT branch_name
FROM account, depositor, customer
WHERE customer.id = depositor.id
    AND depositor.account_number = account.account_number
    AND customer_city = 'Harrison'
```

noahabe rename: added chapter titles to folder names 93005ad · 2 years ago

136 lines (113 loc) · 3.81 KB

Preview Code Blame Raw

> Consider the relational database of Figure 3.19, where the primary keys are underlined. Give an expression in SQL for each of the following queries.
>
> a. Find the ID, name, and city of residence of each employee who works for "First Bank Corporation".
> b. Find the ID, name, and city of residence of each employee who works for "First Bank Corporation" and earns more than $10000.
> c. Find the ID of each employee who does not work for "First Bank Corporation".
> d. Find the ID of each employee who earns more than every employee of "Small Bank Corporation".
> e. Assume that companies may be located in several cities. Find the name of each company that is located in every city in which "Small Bank Corporation" is located.
> f. Find the name of the company that has the most employees (or companies, in the case where there is a tie for the most).
> g. Find the name of each company whose employees earn a higher salary, on average, than the average salary at "First Bank Corporation".

---

a. Find the ID, name, and city of residence of each employee who works for "First Bank Corporation".

```
SELECT e.ID, e.person_name, city
FROM employee AS e, works AS w
WHERE w.company_name = 'First Bank Corporation' AND w.ID = e.ID
```

b. Find the ID, name, and city of residence of each employee who works for "First Bank Corporation" and earns more than $10000.

```
SELECT ID, name, city
FROM employee
WHERE ID IN (
    SELECT ID
    FROM works
    WHERE company_name = 'First Bank Corporation' AND salary > 10000
)
```

This could be written also in the style of the answer to part a, as follows:

```
SELECT e.ID, e.person_name, city
FROM employee AS e, works AS w
WHERE w.company_name = 'First Bank Corporation' AND w.ID = e.ID
    AND w.salary > 10000
```

c. Find the ID of each employee who does not work for "First Bank Corporation".

```sql
SELECT ID
FROM works
WHERE company_name <> 'First Bank Corporation'
```

If one allows people to appear in *employee* without appearing also in *works*, the solution is slightly more complicated. An outer join as discussed in Chapter 4 could be used as well.

```sql
SELECT ID
FROM employee
WHERE ID NOT IN (
    SELECT ID
    FROM works
    WHERE company_name = 'First Bank Corporation'
)
```

d. Find the ID of each employee who earns more than every employee of "Small Bank Corporation".

```sql
SELECT ID
FROM works
WHERE salary > ALL (
    SELECT salary
    FROM works
    WHERE company_name = 'Small Bank Corporation'
)
```

If people may work for several companies and we wish to consider the *total* earnings of each person, the is more complex. But note that the fact that ID is the primary key for *works* implies that this cannot be the case.

e. **Assume that companies may be located in several cities**. Find the name of each company that is located in every city in which "Small Bank Corporation" is located.

```sql
SELECT S.company_name
FROM company AS S
WHERE NOT EXISTS (
    (
        SELECT city
        FROM company
        WHERE company_name = 'Small Bank Corporation'
    )
    EXCEPT
    (
        SELECT city
        FROM company AS T
        WHERE T.company_name = S.company_name
    )
)
```

f. Find the name of the company that has the most employees (or companies, in the case where there is a tie for the most).

```sql
SELECT company_name
FROM works
GROUP BY company_name
HAVING COUNT(DISTINCT ID) >= ALL (
    SELECT COUNT(DISTINCT ID)
    FROM works
    GROUP BY company_name
)
```

g. Find the name of each company whose employees earn a higher salary, on average, than the average salary at "First Bank Corporation".

```sql
SELECT company_name
FROM works
GROUP BY company_name
HAVING AVG(salary) >  (
    SELECT AVG(salary)
    FROM works
    WHERE company_name = 'First Bank Corporation'
)
```